
Frontier & Choosing

Lion, Muon & How to Actually Decide

Lesson 10 of the Optimizer Course

The newest methods — and the practical playbook
for picking and debugging an optimizer

The finale. First, the two newest optimizers worth knowing: Lion (almost insultingly simple) and Muon (setting LLM training-speed records). Then the most useful pages in the whole course: a decision framework for which optimizer to use, a symptom-to-fix debugging table, and honest starting hyperparameters. This is where understanding turns into doing.

1 Lion: the optimizer a computer invented

Lion (“EvoLved Sign Momentum,” 2023) wasn’t designed by hand — it was *discovered* by an automated search over possible algorithms. What it found is shockingly simple. Lion keeps a single momentum buffer, and every step it moves each weight by a *fixed* amount, up or down, depending only on the *sign* of a momentum-blended update:

$$c = \beta_1 m + (1 - \beta_1)g, \quad w \leftarrow w - \eta(\text{sign}(c) + \lambda w), \quad m \leftarrow \beta_2 m + (1 - \beta_2)g$$

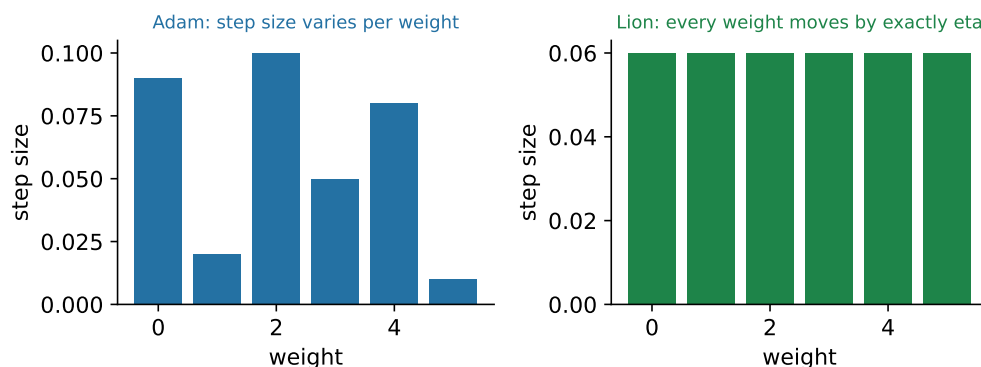
Say it in English: blend momentum with the new gradient, take only its *sign* (+1 or −1), and step every weight by exactly η in that direction (plus weight decay).

Let’s actually do the numbers

With $m = (0.5, -2.0, 0.01)$, $g = (1.0, -0.3, 0.2)$, $\beta_1 = 0.9$:

$$c = 0.9m + 0.1g = (0.55, -1.83, 0.029) \Rightarrow \text{sign}(c) = (+1, -1, +1).$$

Every coordinate moves by exactly η — the magnitude is *always* 1 before scaling. Adam agonises over how big each weight’s step should be; Lion just picks a direction and marches.



Two consequences make Lion attractive at scale: it stores **only momentum** (one buffer per weight, *half* the optimizer memory of Adam’s m -and- v), and it’s competitive with AdamW on large vision and language models. The catch: because every step has unit magnitude, the effective step is larger than it looks, so Lion needs a **smaller learning rate and stronger weight decay** than Adam — retuning is required, not optional.

2 Muon: balance the update

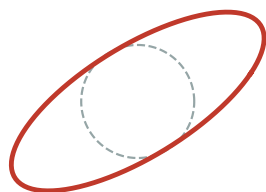
Muon (2024–2025) targets the big 2-D weight *matrices* that make up most of a transformer. It takes the momentum update — which is itself a matrix — and **orthogonalises** it before stepping. Recall from Lesson 7 that a matrix can “act” much more strongly in some directions than others (unequal singular values). Muon evens those out, so the update pushes *equally* in all directions — a cheap stand-in for the whitening that true second-order methods do.

Let's actually do the numbers

A momentum matrix $M = \begin{bmatrix} 2 & 1 \\ 0.5 & 1.5 \end{bmatrix}$ has singular values (2.56, 0.98) — it stretches some directions 2.6× more than others. After orthogonalisation its singular values become (1, 1): the update now acts with equal strength in every direction, instead of dumping most of its effect along one.

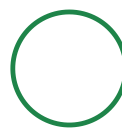
muon. orthogonalize the update so it acts equally in all directions

raw momentum
singular values (2.6, 1.0)



stretches a circle into an ellipse (biased)

orthogonalized
singular values (1, 1)



keeps it a circle (balanced step)

It does this orthogonalisation with a handful of cheap matrix multiplications (a “Newton–Schulz” iteration), avoiding any expensive decomposition. In practice Muon is used *alongside* AdamW — Muon for the hidden 2-D layers, AdamW for embeddings and 1-D parameters — and has been racking up LLM training-speed records. It’s the most exciting recent entrant, but still young: treat it as promising frontier, verify before betting a big run on it.

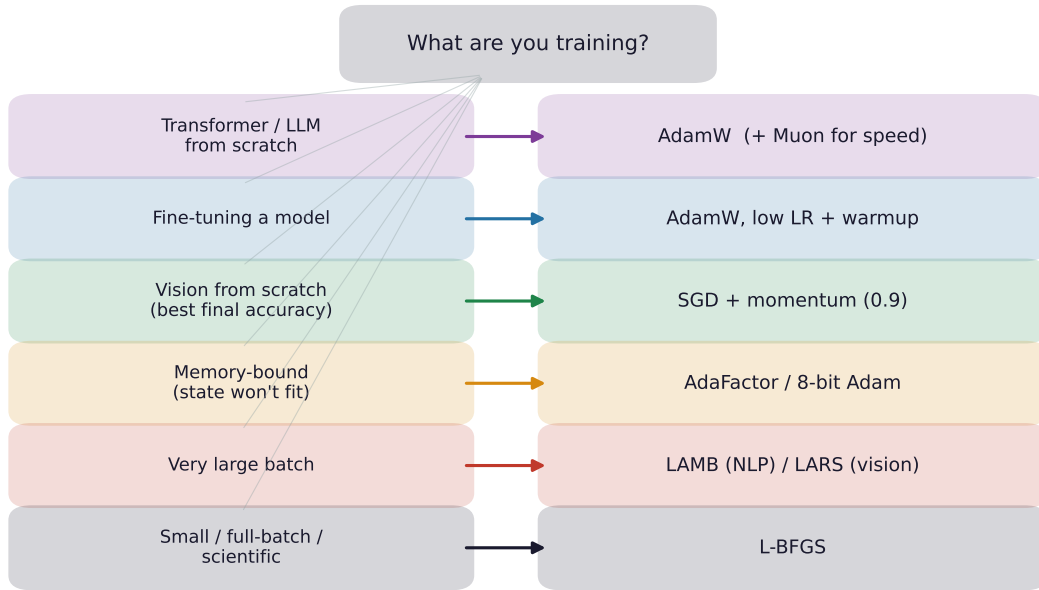
3 The decision: which optimizer should you actually use?

Here’s the part to keep. Match your situation to the recommendation:

- **Default for almost everything: AdamW.** If you’re unsure, this is the answer. It’s robust, well-understood, and rarely the thing that’s wrong.
- **Vision from scratch, chasing best test accuracy: SGD + momentum** ($\beta = 0.9$). Slower and fiddlier, but its noise-driven preference for flat minima (Lesson 8) often wins on final accuracy.
- **Memory-bound: AdaFactor or 8-bit Adam.** When the optimizer state won’t fit.
- **Very large batch: LAMB** (transformers) or **LARS** (vision).
- **Small / full-batch / scientific: L-BFGS.**
- **Chasing training speed on big runs: Muon (+AdamW) or Sophia** — with the caveat that these need engineering care and validation.

4 When it breaks: a symptom-to-fix table

Training misbehaving? Diagnose it here:



Symptom	Likely cause	Fix
Loss $\rightarrow$ NaN / infinity	learning rate too high; no warmup	lower LR; add warmup; clip gradients
Loss jumps around wildly	LR too high for the curvature	lower LR; add momentum
Loss barely moves early	LR too low; Adagrad-style decay	raise LR; switch to Adam/RMSProp
Great on train, poor on test	over-fitting / sharp minimum	add weight decay; try SAM; more data
Slow, steady but crawling	no momentum; wrong optimizer	add momentum; switch to AdamW
Fine, then diverges late	no decay; LR too high at the end	add cosine/linear decay
Diverges in first few steps	unstable start (big init grads)	warmup; smaller initial LR

5 Honest starting points

Reasonable defaults to start from (then tune the learning rate first, everything else second):

- **AdamW:** $\eta = 3 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, weight decay 0.01–0.1.
- **SGD + momentum:** $\eta = 0.1$ (with a step or cosine schedule), momentum 0.9, weight decay $\sim 5 \times 10^{-4}$.
- **Schedule:** linear warmup over the first 2–10% of steps, then cosine decay to near zero.

- **Lion:** learning rate roughly 3–10× *smaller* than your AdamW rate, and weight decay correspondingly *larger*.

Watch out (common confusion)

The most important truth in this entire course: **the optimizer is rarely your bottleneck.** Data quality, model architecture, and the learning rate itself almost always matter more than which optimizer you pick. AdamW with a sane learning rate and a cosine schedule will get you 95% of the way on almost any problem. Spend your energy there before agonising over Lion vs Sophia vs Muon. Reach for the exotic options only when you have a specific, diagnosed reason — a memory wall, a batch-size wall, a generalisation gap, a speed target.

6 The whole course, in one breath

You started by learning to “feel” a hill: the *gradient* (slope), the *Hessian* (curvature), the *condition number* (how lopsided), and the *Taylor* idea that any hill is a bowl up close. Then every optimizer was one answer to “how much curvature can I afford?”:

- **GD / SGD** (Lessons 2): ignore curvature, just step downhill.
- **Momentum** (Lesson 3): remember past steps to smooth the zig-zag.
- **Adagrad / RMSProp** (Lesson 4): cheap per-weight curvature guess.
- **Adam / AdamW** (Lesson 5): momentum + per-weight scaling — the default.
- **AdaFactor / LAMB / LARS** (Lesson 6): the same, made to survive billions of weights.
- **Newton / K-FAC / Shampoo / Sophia** (Lesson 7): use the real curvature, cleverly approximated.
- **SAM / Lookahead** (Lesson 8): aim for *flat* bottoms, which generalise.
- **Schedules** (Lesson 9): change the step size over time — the orthogonal knob.
- **Lion / Muon** (Lesson 10): the newest tricks — a sign step, an orthogonal step.

Every one of them is the same blindfolded walk down the same foggy hill — just with smarter and smarter ways of deciding which way to step, and how far. You now understand all of them, from the arithmetic up. Go train something.

End of the Optimizer Course. You can build, run, and reason about every major optimizer in deep learning — and, more importantly, choose the right one and fix it when it breaks.